CAMPUS CAFE

A Full-Stack Point-of-Sale System for Campus Dining

NEU 5001 — Final Project

Built with Python • FastAPI • React

Presented by:

Arja Sadhukhan
Yanyan Wang

Agenda

- Project Overview — What we built and why
- System Architecture — How the pieces fit together
- The Main Function — CLI application flow and thought process
- Class Design Deep Dive — Menu, Order, Inventory, Customer, Staff
- Data Persistence — Excel-based tracking system (Special Feature)
- Frontend Application — React web interface
- Live Demo & Q&A

Project Overview

What is Campus Cafe?

Campus Cafe is a point-of-sale (POS) system for a campus dining environment, supporting customer ordering and staff inventory management. It runs on the terminal AND as a modern web application.

Customer Features

- Browse menu by category (Salads, Drinks, Desserts, Sandwiches)
- Add / remove items from cart with quantity control
- Daily special with automatic 20% discount
- Checkout with loyalty points & printed receipt

Staff Features

- Secure login with ID + password
- View real-time inventory levels
- Restock items in bulk
- Session summary on quit (all orders + total revenue)

Dual Interface

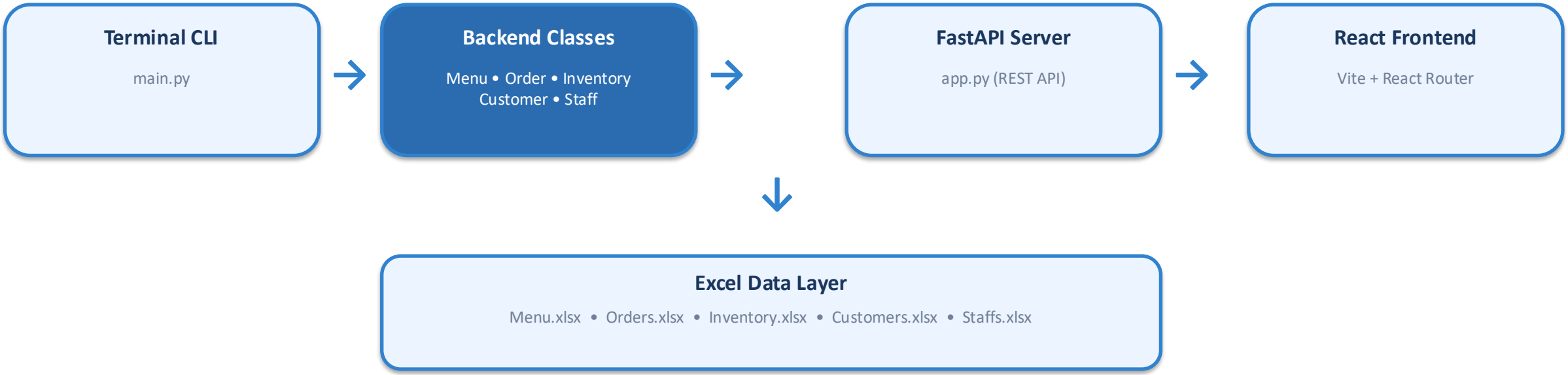
- Terminal-based CLI application (main.py)
- React web application (frontend/)
- Both interfaces share the exact same backend classes

Data Persistence

- All data stored in Excel (.xlsx) files
- Orders, customers, inventory, staff records
- Loyalty points accumulate across sessions

System Architecture

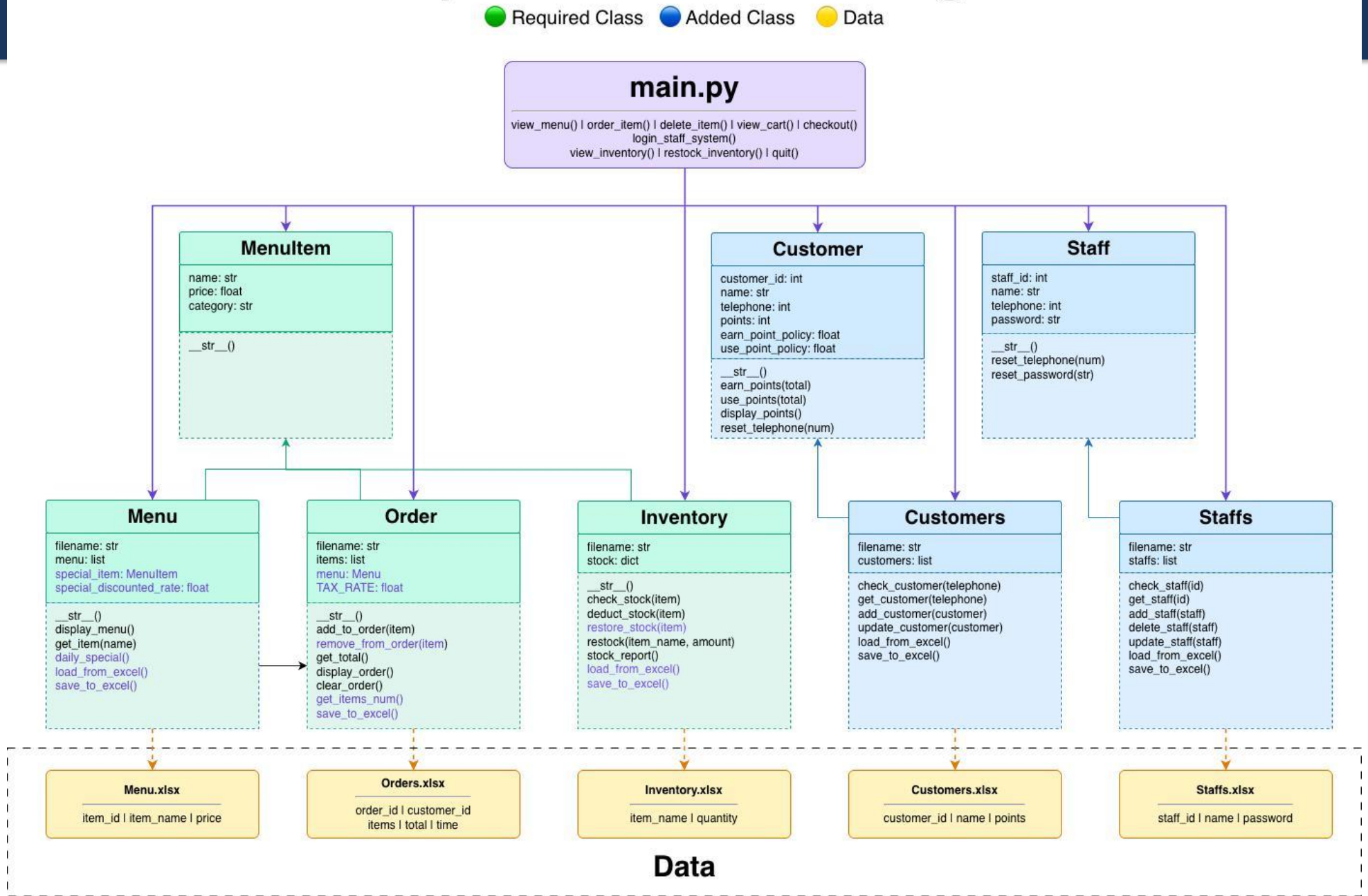
How the pieces fit together



Design Principle

The backend classes are shared between the CLI and the web app. The FastAPI server wraps the same Python classes that main.pyuses, ensuring consistent behavior across both interfaces.

Campus Café — Class Diagram



SECTION 1

The Main Function

CLI application flow and thought process

Main Function — Application Flow

main.py: the entry point that orchestrates everything

Thought Process: We designed main.py as a menu-driven loop. The user sees numbered options, picks one, and the program delegates to a dedicated function. This keeps each feature isolated and the main loop clean.

ENTRY POINT & GLOBAL STATE

```python
from backend.menu import MenuItem, Menu
from backend.order import Order
from backend.inventory_ShuranRyu import Inventory
from backend.customer import Customer, Customers
from backend.staff import Staff, Staffs
from getpass import getpass
from datetime import datetime

MENU = Menu() #Load menu from Excel
order = Order(MENU) #Current order
staffs = Staffs() #Load staff credentials
session_orders = {} #Track all orders
```

Why This Design?

Each function (view_menu, order_item, checkout, etc.) is self-contained with its own input loop and validation. Global state (MENU, order, session_orders) is initialized once at the top and shared across functions. This mirrors how a real POS works— one session, one shared order, one menu.

Main Function — Application Flow

main.py: the entry point that orchestrates everything

Thought Process: We designed main.py as a menu-driven loop. The user sees numbered options, picks one, and the program delegates to a dedicated function. This keeps each feature isolated and the main loop clean.

MAIN_MENU() — THE DRIVING LOOP

```
print(
    f"{'[1]': >13}  {'VIEW MENU': <23}\n\n"
    f"{'[2]': >13}  {'ORDER ITEM': <23}\n\n"
    f"{'[3]': >13}  {'VIEW CART': <23}\n\n"
    f"{'[4]': >13}  {'CHECKOUT': <23}\n\n"
    f"{'[s]': >13}  {'STAFF MODE (BONUS)': <23}\n\n"
    f"{'[q]': >13}  {'QUIT': <23}\n\n"
)
```

```
if choice == "1": view_menu()
elif choice == "2": order_item()
elif choice == "3": view_cart()
elif choice == "4": checkout()
elif choice == "q": quit()
elif choice == "s" or choice == "staff": login_staff_system()
else: print("Invalid input. Please enter 1, 2, 3, 4, s, or q.")
```

Why This Design?

Each function (view_menu, order_item, checkout, etc.) is self-contained with its own input loop and validation. Global state (MENU, order, session_orders) is initialized once at the top and shared across functions. This mirrors how a real POS works— one session, one shared order, one menu.

Main Function — Ordering & Checkout

Input validation, stock checking, and receipt generation

ORDER_ITEM() — WITH STOCK VALIDATION

```python
def order_item():
    while True:
        item_name = input("Enter the name of food you would like to order or R to return to homepage: ").lower().strip()

        if item_name == "r":
            return

        item = MENU.get_item(item_name)

        if item is None:
            print("Invalid input, please try again!")
            continue

        while True:
            num = input(f"Enter the number of {item.name} you would like to order: ")

            try:
                num_int = int(num)
                if num_int <= 0:
                    print("Invalid input, please try again!")
                    continue
                break
            except ValueError:
                print("Invalid input, please try again!")
                continue
```

```python
inventory = Inventory()
counter = 0
for i in range(num_int):
    if inventory.check_stock(item):
        counter += 1
        order.add_to_order(item)
        inventory.deduct_stock(item)
        inventory.save_to_excel()
    else:
        print("Sorry, this item is out of stock, please choose other item.")
        break
print(f"You have ordered {counter} {item.name}.")
```

Key Decision: Per-Unit Stock Check

We check stock per unit in a loop rather than checking total availability upfront. This enables partial fulfillment — if 3 are requested but only 2 remain, the customer gets 2 instead of 0.

Main Function — Ordering & Checkout

Input validation, stock checking, and receipt generation

CHECKOUT() — RECEIPT & LOYALTY POINTS

```
def checkout():
    if len(order.items) == 0:
        print("Sorry, you didn't order anything.")
        return

    else:
        # Calculate total consuming
        subtotal = order.get_total()
        total_before_tax = order.get_total_after_discount()
        discount = subtotal - total_before_tax
        tax = order.TAX_RATE * total_before_tax
        total = total_before_tax + tax

        # Get valid telephone number
        while True:
            telephone = input("Enter your telephone number: ")

            try:
                telephone_int = int(telephone)
                if telephone_int <= 0:
                    print("Invalid input, please try again.")
                    continue
                break
            except ValueError:
                print("Invalid input, please try again.")
```

```
    # Earn loyalty points
    customers = Customers()
    if customers.check_customer(telephone_int):
        customer = customers.get_customer(telephone_int)
        customer.earn_points(total)
        customers.update_customer(customer)
    else:
        customer = Customer(telephone_int)
        customer.earn_points(total)
        customers.add_customer(customer)

    customers.save_to_excel()
```

Key Decision: Auto Customer Registration

At checkout, we check if the phone number exists. Returning customers earn more points; new customers are automatically registered. No signup flow needed — frictionless loyalty.

SECTION 2

Class Design Deep Dive

Object-oriented design for Menu, Order, Inventory, Customer, and Staff

Class: Menu & MenuItem

backend/menu.py — Menu management with daily specials

Design Decisions

- MenuItem is a simple data class — name, price, category
- Menu loads all items from Menu.xlsx on initialization
- Daily Special randomly chosen via random.choice() each session, with configurable discount rate (default 20%)
- Fuzzy search: get_item() ignores case and spaces, so "fruitsalad" matches "Fruit Salad"

KEY METHODS

```
class MenuItem:
    """
    Represents a single menu item with name, price, and category.
    """

    def __init__(self, name, price, category):
        """
        Initialize a MenuItem.

        Args:
            name (str): the name of the item
            price (float): the price of the item
            category (str): the category (e.g. "Drinks", "Desserts")
        """

        self.name = name
        self.price = price
        self.category = category
```

Class: Menu & MenuItem

backend/menu.py — Menu management with daily specials

Design Decisions

- MenuItem is a simple data class — name, price, category
- Menu loads all items from Menu.xlsx on initialization
- Daily Special randomly chosen via random.choice() each session, with configurable discount rate (default 20%)
- Fuzzy search: get_item() ignores case and spaces, so "fruitsalad" matches "Fruit Salad"

```python
class Menu:
    """
    Represents the full cafe menu, supporting display, search, and plotting.
    """

    def __init__(self, filename="Menu.xlsx", rate = 0.2):
        """
        Initialize the Menu.

        Args:
            menu (list): a list of MenuItem objects
        """
        self.filename = filename
        self.menu = self.load_from_excel()
        self.special_item = self.daily_special()
        self.special_discounted_rate = rate
```

KEY METHODS

```python
def __str__(self):
    """
    Return formatted string: index, name, and price.

    Returns:
        str: formatted string like "1 Americano          $5.00"
    """
    return f"{self.name: <34}${self.price: >5.2f}\n"
```

Class: Menu & MenuItem

backend/menu.py — Menu management with daily specials

KEY METHODS

```
def display_menu(self):
    """
    Print the menu to the console, grouped by category.
    """
    print(self)
```

```
def get_item(self, name):
    """
    Search for a menu item by name (case-insensitive, ignores spaces).

    Args:
        name (str): the name of the item to search for

    Returns:
        MenuItem or None: the matching item, or None if not found
    """
    for item in self.menu:
        if item.name.lower().strip().replace(" ", "") == name.lower().strip().replace(" ", ""):
            return item
    return None
```

Class: Menu & MenuItem

backend/menu.py — Menu management with daily specials

KEY METHODS

```
MENU_MANAGER = Menu()
MENU = MENU_MANAGER.menu

def display_menu(menu):
    print(_format_menu_items(menu))

def get_item(menu, name):
    normalized_name = _normalize_name(name)
    for item in menu:
        if _normalize_name(item.name) == normalized_name:
            return item
    return None
```

Class: Order

backend/order.py — Cart management, pricing, and tax calculation

Design Decisions

- Items stored as a list (not dict) — each instance individually, making add/remove simple
- Discount logic in get_total_after_discount() — only the daily special gets the discount
- Tax rate is a class constant (0.1 = 10%), easy to adjust
- save_to_excel() auto-increments order_id and appends to Orders.xlsx with timestamps

KEY METHODS

```
class Order:
    """
    Represents a customer's order, storing items and handling checkout.
    """

    def __init__(self, menu, filename = "Orders.xlsx"):
        self.filename = filename
        self.items = []   # list of MenuItem objects in the order
        self.menu = menu
        self.TAX_RATE = 0.1 # 10% tax rate


    def __str__(self):
        """
        Return a formatted string of the order with item names, quantities, and prices.

        Returns:
            str: formatted order string with item names, quantities, and prices
        """
        order =  f"{'-' * 40}\n  {'Item': <26}{'Num'}{'Price': >8}\n{'-' * 40}\n"
```

Class: Order

backend/order.py — Cart management, pricing, and tax calculation

KEY METHODS

```python
def add_to_order(self, item):
    """
    Add a MenuItem to the order.

    Args:
        item (MenuItem): the menu item to add
    """
    self.items.append(item)

def remove_from_order(self, item):
    """
    Remove one instance of a MenuItem from the order.

    Args:
        item (MenuItem): the menu item to remove
    """
    self.items.remove(item)
```

```python
def get_total(self):
    """
    Calculate and return the subtotal (before tax) of all items.

    Returns:
        float: the subtotal price before tax
    """
    total = 0.00
    for item in self.items:
        total += item.price
    return total

def get_total_after_discount(self):
    """
    Calculate and return the subtotal (before tax) of all items.

    Returns:
        float: the subtotal price before tax
    """
    total = 0.00
    for item in self.items:
        if item == self.menu.special_item:
            total += item.price * (1-self.menu.special_discounted_rate)
        else:
            total += item.price
    return total
```

Class: Order

backend/order.py — Cart management, pricing, and tax calculation

KEY METHODS

```python
def display_order(self):
    """
    Print the order summary to the console.
    """
    print(self)

def clear_order(self):
    """
    Clear all items from the order.
    """
    self.items = []

def get_items_num(self):
    item_counter = {}

    # Count the quantity of each item
    for item in self.items:
        if item not in item_counter.keys():
            item_counter[item] = 1
        else:
            item_counter[item] += 1

    return item_counter
```

```python
def save_to_excel(self):
    if len(self.items) == 0:
        return

    order_path = data_file(self.filename)
    if order_path.exists():
        df_old = pd.read_excel(order_path)
        order_id = 1 if df_old.empty else int(df_old["order_id"].max()) + 1
    else:
        df_old = None
        order_id = 1
```

Class: Inventory

backend/inventory_ShuranRyu.py — adapted from Shuran and Ryu's Week 1 inventory class

Why We Chose This Class

We selected Shuran and Ryu's Inventory class because it followed the shared contract closely and kept the stock logic simple: a dictionary keyed by item name plus clear methods for checking, deducting, and restocking inventory. That made it the easiest Week 1 class to trust, extend, and wire into our Week 2 app controller.

What We Reused

- check_stock(item) returns a clean True/False guard before ordering
- deduct_stock(item) reduces one unit at a time for accurate quantity loops
- restock(name, amount) already matched the staff restock workflow
- stock_report() gave us a solid foundation for the staff inventory view

COLLABORATIVE INTEGRATION

```
class Inventory:
    """
    Inventory class adapted from Shuran and Ryu's
    Week 1 implementation.
    """
    def __init__(self, filename="Inventory.xlsx"):
        self.filename = filename
        self.stock = self.load_from_excel()

    def __str__(self):
        inventory = f"{'-' * 40}\n"
        for item_name, quantity in self.stock.items():
            inventory += f"* {item_name: <25}{int(quantity): >13d}\n"
        inventory += f"{'-' * 40}\n"
        return inventory
```

Class: Inventory

backend/inventory_ShuranRyu.py — compatibility changes for our full app

Compatibility Changes We Made

Shuran and Ryu's original version used a hard-coded stock dictionary and a local test Item class. We preserved the required stock contract, then connected inventory to real MenuItem objects, Excel persistence, restore-on-delete behavior, and the shared data_file() helper used by both the CLI and the deployed app.

COMPATIBILITY LAYER

```
def check_stock(self, item):
    item_quantity = self.stock.get(item.name, 0)
    return item_quantity > 0

def deduct_stock(self, item):
    if self.check_stock(item):
        self.stock[item.name] -= 1
```

```
def restore_stock(self, item):
    self.stock[item.name] = self.stock.get(item.name, 0) + 1

def restock(self, item_name, amount):
    self.stock[item_name] = self.stock.get(item_name, 0) + amount

def stock_report(self):
    print(f"{'INVENTORY': ^40}")
    print(self)
```

Class: Inventory

backend/inventory_ShuranRyu.py — debugging lessons and integration challenges

Challenges We Faced

The hardest part was interface mismatch and state synchronization. The original code printed stock messages directly, storeddata only in memory, and assumed a local data/ path. We moved user-facing messages into the controller/API, matched stock keys to MenuItem.name, restored stock on cart deletion, and centralized file access through data_file().

ADAPTATION & DEBUGGING

```
def load_from_excel(self):
    inventory_path = data_file(self.filename)
    if not inventory_path.exists():
        return {}

    df = pd.read_excel(inventory_path)
    result = {}
    for _, row in df.iterrows():
        result[row["Item Name"]] = int(row["Stock Numbers"])
    return result

def save_to_excel(self):
    df = pd.DataFrame(
        list(self.stock.items()),
        columns=["Item Name", "Stock Numbers"],
    )
    df.to_excel(data_file(self.filename), index=False)
```

Inventory Integration — What Broke And What We Fixed

Original Assumptions

- Stock lived in a hard-coded dictionary
- A local Item test class stood in for real menu items
- check_stock() handled user messaging itself
- No Excel persistence or cart-delete restore path

Integration Problems

- main.py and app.py needed silent True/False stock checks
- Deleting cart items had to put stock back immediately
- CLI and frontend both needed the same inventory source
- Raw data/ paths were fragile in deployment

Changes We Made

- Imported the real Menu/MenuItem workflow
- Added restore_stock(item), load_from_excel(), and save_to_excel()
- Routed all file access through data_file()
- Saved inventory after order, delete, and restock actions

Why It Matters

- We extended the reused class without breaking its public contract methods
- That made the inventory code judge-ready for Week 2 integration and Week 3 edge-case testing

Contract Impact

The final version still exposes check_stock(item), deduct_stock(item), restock(item_name, amount), and stock_report(), but now it also supports persistence, cart deletion, and the public frontend.

Class: Customer & Customers

backend/customer.py — Loyalty program with automatic point tracking

Design Decisions

- Auto-generated IDs: class-level counter produces unique 9-digit IDs (e.g., 000000001)
- Phone as identifier: customers looked up by phone number; duplicate prevention built in
- Points policy: earn 1 point per $5 spent — clear, simple, configurable
- Customers class manages the collection — CRUD operations + Excel persistence
- Counter syncs with max existing ID on load to prevent duplicates

KEY METHODS

```python
class Customer:
    counter = 0
    def __init__(self, telephone, name = "", points = 0, id = None):
        if id != None:
            self.customer_id = id
            Customer.counter = max(Customer.counter, int(id))
        else:
            Customer.counter += 1
            self.customer_id = str(Customer.counter).zfill(9)

        if name != "":
            self.name = name;
        else:
            self.name = ''.join(random.choices(string.ascii_letters + string.digits, k=9))

        self.telephone = telephone
        self.points = points
        self.earn_point_policy = 5
        self.use_point_policy = 1
```

Class: Customer & Customers

backend/customer.py — Loyalty program with automatic point tracking

KEY METHODS

```python
def __str__(self):
    return f"{self.customer_id}  {self.name}  {self.telephone}  {self.points:.2f}\n"

def earn_points(self, consuming):
    self.points += consuming//5

def use_points(self, consuming):
    self.points -= consuming * 1

def display_points(self):
    print(self)

def reset_name(self, name):
    self.name = name

def reset_telephone(self, num):
    self.telephone = num
```

```python
class Customers:
    def __init__(self, filename='Customers.xlsx'):
        self.filename = filename
        self.customers = self.load_from_excel()

    def check_customer(self, telephone):
        for customer in self.customers:
            if customer.telephone == telephone:
                return True
        return False

    def get_customer(self, telephone):
        for customer in self.customers:
            if customer.telephone == telephone:
                return customer
        return None

    def add_customer(self, customer):
        if not self.check_customer(customer.telephone):
            self.customers.append(customer)

    def update_customer(self, customer):
        for i in range(len(self.customers)):
            if self.customers[i].customer_id == customer.customer_id:
                self.customers[i] = customer
```

Class: Customer & Customers

backend/customer.py — Loyalty program with automatic point tracking

KEY METHODS

```
def load_from_excel(self):
    customer_path = data_file(self.filename)
    if not customer_path.exists():
        return []

    df = pd.read_excel(customer_path)
    result = []
    for _, row in df.iterrows():
        result.append(
            Customer(
                row["telephone"],
                row["name"],
                row["points"],
                str(row["customer_id"]).zfill(9),
            )
        )
        Customer.counter = max(Customer.counter, int(row["customer_id"]))
    return result
```

Class: Staff & Staffs

backend/staff.py — Secure staff authentication and management

Design Decisions

- Same ID pattern as Customer — 9-digit auto-increment with class-level counter
- Password-protected: staff must enter both ID and password to access
- In the CLI, getpass hides password input for security
- Staffs collection mirrors the Customers pattern: check, get, add, delete, update + Excel I/O

Consistent Patterns

Customer and Staff follow the same architectural template — entity class + collection class + Excel persistence. This makes the codebase predictable and easy to extend.

STAFF LOGIN FLOW (MAIN.PY)

```
def login_staff_system():
    while True:
        id = input("Enter your staff id or R to return to homepage: ").lower()

        if id == "r":
            return

        if not staffs.check_staff(id):
            print("Invalid input, please try again.")
            continue
        else:
            staff = staffs.get_staff(id)

            while True:
                pwd = getpass("Enter the password or R to return to homepage:")
                print()


                if pwd == "r":
                    return

                elif pwd == staff.password:
                    staff_menu()
                    return

                else:
                    print("Invalid input, please try again!")
```

Class: Staff & Staffs

backend/staff.py — Secure staff authentication and management

Design Decisions

- Same ID pattern as Customer — 9-digit auto-increment with class-level counter
- Password-protected: staff must enter both ID and password to access
- In the CLI, getpass hides password input for security
- Staffs collection mirrors the Customers pattern: check, get, add, delete, update + Excel I/O

Consistent Patterns

Customer and Staff follow the same architectural template — entity class + collection class + Excel persistence. This makes the codebase predictable and easy to extend.

SESSION QUIT — SUMMARY ON EXIT

```
def quit():
    print("\n"*10)
    print(f"{'CAMPUS CAFÉ': ^40}")
    session_orders_total = 0
    for _, info in session_orders.items():
        session_orders_total += info["Total"]
        print(info["Items"])


    print(f"{len(session_orders)} orders completed, {session_orders_total:.2f} dollars earned.")

    exit()

if __name__ == "__main__":

    main_menu()
```

SECTION 3

Data Persistence

Excel-based tracking for orders, customers, and inventory

Excel Data Layer

All data persisted to .xlsx files using pandas + openpyxl

File	Purpose	Columns
Menu.xlsx	Menu items catalog	item_name, price, category
Orders.xlsx	Complete order history	order_id, total, date, time, [item quantities]
Inventory.xlsx	Current stock levels	Item Name, Stock Numbers
Customers.xlsx	Customer profiles & loyalty points	customer_id, name, telephone, points
Staffs.xlsx	Staff credentials	staff_id, name, telephone, password

★ Special Feature: Loyalty Points Tracking

Every checkout automatically identifies returning customers by phone number, accumulates their loyalty points in Customers.xlsx, and displays the updated balance on the receipt. New customers are auto-registered on first purchase with a unique 9-digit ID. All data persists across sessions — no data is lost when the program restarts.

SECTION 4

Frontend Application

A React web interface built on top of the same backend

Test out our Live app here:

Scan the QR code or click on the link to check out our live frontend app:

QR Code:

Publicly hosted URL for our Campus Café web app:

https://campus-cafe-cs5001-production.up.railway.app/

React Frontend

Modern web interface for the Campus Cafe POS

To make our project more accessible and visually engaging, we built a full web application using React. A FastAPI server (app.py) wraps the exact same backend classes, exposing them as REST API endpoints.

Tech Stack

- React 18 + Vite
- React Router v6
- FastAPI (Python)
- Vite proxy for API calls

Customer Pages

- Home — landing page
- Menu — browse by category
- Cart — review & edit quantities
- Checkout — pay & get receipt

Staff Pages

- Staff Login — secure auth
- Inventory Management
- Restock items in bulk
- Session summary on quit

Same Backend, Two Interfaces

Both the terminal CLI and the React frontend call the same Python classes. The web app adds a visual layer on top without duplicating any business logic. This demonstrates clean separation of concerns — the backend handles data and rules, while the frontend handles presentation.

Frontend — Key Features

Interactive UI with real-time stock validation

Menu Page

- 4 category tabs (Salads, Drinks, Desserts, Sandwiches)
- Daily special banner with discount badge
- Inline quantity controls (+/-) with stock validation
- Toast notifications when stock is insufficient

Cart Page

- Editable quantities (type or +/- buttons)
- Real-time price summary with tax & discount
- Stock validation on quantity increase
- Clear cart functionality

Checkout & Receipt

- Phone number entry for loyalty points
- Full receipt with itemized breakdown
- Auto-redirect to home after 8 seconds
- Automatic customer registration

Staff Dashboard

- Inventory table with low-stock highlighting (red)
- Restock form with dropdown item selection
- Navbar "Quit" button triggers session summary
- Modal shows all orders with item details + revenue

Edge Cases & Error Handling

Empty Cart Checkout

- checkout() first checks whether order.items is empty. If so, it prints a friendly message and returns to the main menu instead of producing a $0 receipt or crashing.

Item Not Found

- MENU.get_item() returns None for unknown names. order_item() catches that case, shows an error, and re-prompts the user without changing stock or cart state.

```
Please enter your choice: 4
Sorry, you didn't order anything.
```

```
Please enter your choice: Tres Leches Cake
Invalid input. Please enter 1, 2, 3, 4, or q.
```

Why This Matters

These fixes map directly to the CS5001 Student Guide requirements for Week 3. We tested them in the terminal app so the demoflow stays stable even when the user makes mistakes.

Edge Cases & Error Handling

Out of Stock

- Inventory.check_stock() blocks unavailable items. If stock is 0, the app prints a clear message, does not deduct anything, and lets the customer choose a different item.

```
Enter the name of food you would like to order or R to return to homepage: Chicken Sandwich
Enter the number of Chicken Sandwich you would like to order: 10
Sorry, this item is out of stock, please choose other item.
You have ordered 0 Chicken Sandwich.
Enter the name of food you would like to order or R to return to homepage:
```

Why This Matters

These fixes map directly to the CS5001 Student Guide requirements for Week 3. We tested them in the terminal app so the demoflow stays stable even when the user makes mistakes.

Edge Cases & Error Handling

Invalid Inputs

- main_menu() rejects anything other than 1, 2, 3, 4, q, or staff. We also validate quantity, telephone, delete, and restock inputs so non-numeric or non-positive values never break the session.

```
            CAMPUS CAFÉ

        [1]  VIEW MENU

        [2]  ORDER ITEM

        [3]  VIEW CART

        [4]  CHECKOUT

        [q]  QUIT


Please enter your choice: 41
Invalid input. Please enter 1, 2, 3, 4, or q.
```

```
Enter the name of the food you would like to restock or R to return to staff menu: 3546
Invalid input, please try again!
Enter the name of the food you would like to restock or R to return to staff menu: Gorilla
Invalid input, please try again!
Enter the name of the food you would like to restock or R to return to staff menu:
```

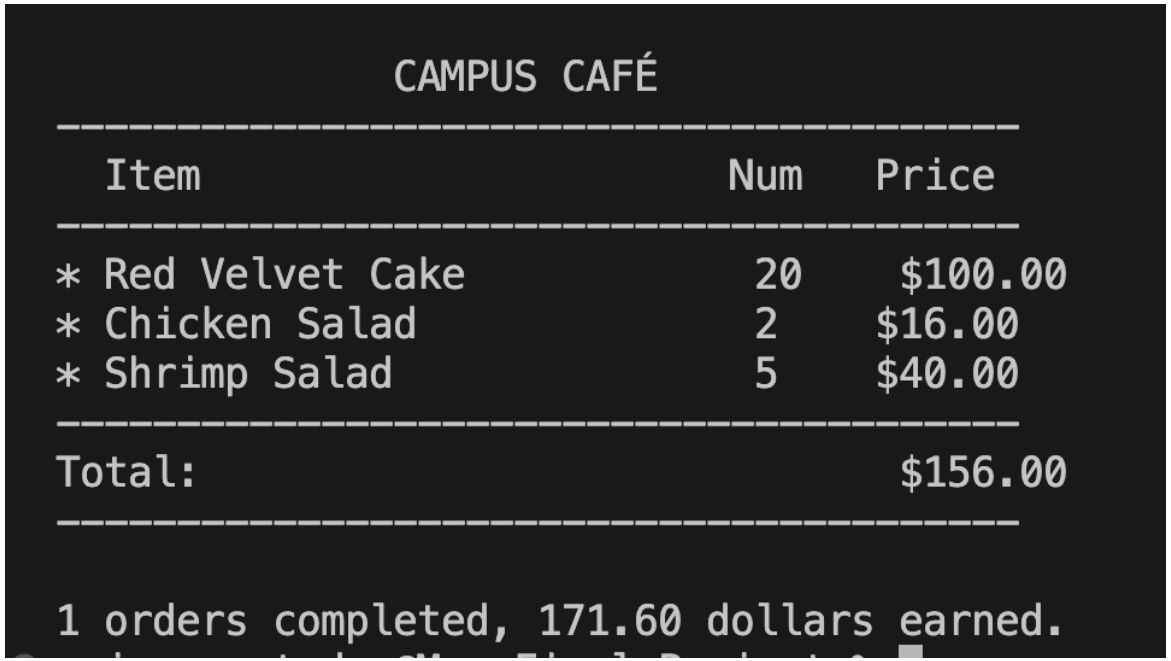
```
Enter the name of the food you would like to restock or R to return to staff menu: Eclair
Enter the number of Eclair you would like to restock: -50
Invalid input, please try again!
Enter the number of Eclair you would like to restock: T67
Invalid input, please try again!
Enter the number of Eclair you would like to restock:
```

```
Please enter your choice: 4
Enter your telephone number: ihjjkl
Invalid input, please try again.
Enter your telephone number: 2345678890
Money received.
```

Session Summary

Session Summary

- Session summary displays after the Staff quits the Staff mode. It shows the number of orders completed and the total revenue generated for the session.



Bonus Features & Code Improvements

Bonus Features

- Daily special with an automatic 20% discount, a loyalty-points system, staff restock mode, full session summaries, and a React frontend deployed to a public link all extend the base brief.

Persistence & Full Stack

- Orders, customers, staff, menu items, and inventory are stored in Excel and reused by both the CLI and the FastAPI + React frontend, so we did not duplicate business logic between interfaces.

Code Improvements

- We improved contract alignment, fixed test blocks, added clearer helper functions, and documented non-obvious logic such as daily-special pricing, inventory restoration, and deployment-safe data loading.

Public Demo Ready

- The frontend is now publicly hosted, and the presentation includes a QR code so judges can try the web interface directly during or after the talk.

Team Reflection

Our biggest growth came from integration work: comparing classmates' code, adapting it carefully instead of rewriting it, andthen extending a terminal app into a deployable full-stack project.

Frontend Staff Demo Access

Access Steps

- Open the live Campus Cafe link and click the Staff tab in the top navigation bar. The app takes you to /staff/login, where you can sign in and continue directly to the staff dashboard.

Staff Demo Credentials

- Primary: 000000001 / demo123
- Backup: 000000002 / demo456

What To Review

- After login, judges can view the inventory table, see low-stock highlighting, restock items in bulk, and use the Quit button to display the staff session summary.

Synced Credentials

- These demo credentials are now the same in both our local project data and the public hosted frontend, so the same login works in both environments.

Live App

Public URL: https://campus-cafe-cs5001-production.up.railway.app

Thank You

Questions & Live Demo

Campus Cafe — NEU 5001 Final Project

Python • FastAPI • React • Excel